Rapport de stage :

Étude des modèles Transformer appliqués au problème SAT

Hélios Gilquin

Licence 3 Informatique - Parcours MIAGE

Année 2024-2025

Sommaire

Contexte et objectifs....3
Présentation générale du Transformer....3
1. Qu'est ce qu'un Transformer ?....3
2. Fonctionnement d'un Transformer....3
Exemple de pipeline....3
Tokenization....4
Embeddings....4
Ajout du Positional Encoding....4
Self-Attention....4
Étape 1 - Création des vecteurs Q, K, V (Représentation contextuelle initiale)....4
Étape 2 - Calcul des scores d'attention....5
Feed forward....5
Add & Norm....6
Multi-Head Attention....6
Encodeur / Décodeur Blocs....6
Génération de la sortie (Pour mon modèle)....7
Comment entraîner un Transformer pour résoudre le SAT....7
Génération de données d'entraînement....7
Entrainement du Transformer....8
Utilisation du modèle entraîné....8
Conclusion et perspective....9
Conclusion....9
Perspective....9

Contexte et objectifs

Le problème SAT (satisfiabilité booléenne) est un problème fondamental en informatique. Il consiste à déterminer si une formule logique peut être rendue vraie en attribuant des valeurs aux variables qui la composent.

Les modèles Transformer, initialement développés pour le traitement du langage naturel, se distinguent par leur mécanisme d'attention, capable de capturer des relations complexes au sein de séquences. Ce mécanisme est aujourd'hui au cœur des grands modèles de langage (LLM).

Ce stage explore le potentiel des modèles Transformer pour résoudre le problème SAT, en les utilisant comme solveurs approximatifs. Il vise également à évaluer leur complémentarité avec des méthodes classiques exactes telles que l'algorithme DPLL. L'objectif est de mieux cerner les capacités et les limites des Transformers en modélisation logique, tout en approfondissant la compréhension de leur fonctionnement interne.

Présentation générale du Transformer

1. Qu'est ce qu'un Transformer ?

Un Transformer est une architecture de modèle d'apprentissage automatique, utilisée en intelligence artificielle, spécialement conçue pour traiter des séquences de données, comme le langage naturel.

Il repose sur un mécanisme appelé attention, qui permet au modèle de prendre en compte l'importance relative de chaque élément d'une séquence, quel que soit son ordre, ce qui améliore considérablement la compréhension du contexte.

Introduit en 2017 par l'article *"Attention is All You Need"*, le Transformer a remplacé les architectures séquentielles traditionnelles.

Les Transformers sont à la base des grands modèles de langage comme GPT, BERT ou T5, qui sont capables de comprendre, générer et traduire du texte avec un haut niveau de performance. Ils représentent aujourd'hui une révolution dans l'IA moderne.

2. Fonctionnement d'un Transformer

Exemple de pipeline :

Input -> Tokenization -> Embeddings + positional encoding -> Bloc encodeur et/ou décodeur (Self-attention, etc…) -> Output

Tokenization :

Le tokenizer transforme le texte en une séquence de tokens (souvent des sous-mots ou morceaux de mots), Elle dépend de la liste de vocabulaire du Transformer.

Chaque token est ensuite associé à un ID unique correspondant à sa position dans le vocabulaire du modèle.

Embeddings :

On peut voir la matrice d'embedding comme un dictionnaire géant : au lieu de donner une définition en mots, elle fournit une représentation vectorielle des tokens.

Ces vecteurs, appelés embedding vectors, ont une dimension fixe et capturent des propriétés sémantiques et syntaxiques du token

Attention, ces vecteurs encodent une signification de base du token, mais hors contexte (on donne un contexte plus tard).

Les embeddings sont appris automatiquement pendant l'entraînement du Transformer.

Cette étape n'a lieu que dans le tout premier bloc encodeur. Les ID des tokens (fournit dans l'étape d'avant) servent d'index pour récupérer leur vecteur dans la matrice d'embedding.

Ajout du Positional Encoding :

Contrairement aux modèles séquentiels, les Transformers ne traitent pas les mots dans l'ordre. Ils voient toute la séquence en parallèle, grâce au mécanisme d'attention.

Pour introduire la notion de position, on ajoute un encodage de position (positional encoding) : un vecteur qui dépend de la position du token dans la phrase.

L'embedding final est donc obtenu en additionnant le vecteur d'embedding et l'encodage de position.

Self-Attention :

C'est la partie la plus importante d'un modèle Transformer. Ce mécanisme permet à chaque token de "regarder" tous les autres tokens pour comprendre le contexte, et leur accorder un poids en fonction de leur pertinence dans la séquence. Elle se trouve dans les blocs encodeurs et décodeurs.

Étape 1 - Création des vecteurs Q, K, V (Représentation contextuelle initiale)

Chaque vecteur d'embedding est transformé en trois vecteurs distincts à l'aide de matrices de poids apprises pendant l'entraînement : Wq, Wk et Wv. Ces matrices permettent de représenter les embeddings sous forme de requêtes (query), clés (key) et valeurs (value). Ainsi, chaque token ajuste sa représentation vectorielle en fonction de son contexte, c'est-à-dire selon son entourage dans la phrase.

Voici les trois vecteurs générés :

- **Q (Query)** - Ce que je veux savoir.
- **K (Key)** - Ce que je propose comme information.
- **V (Value)** - Ce que je réponds si je suis interrogé.

Les vecteurs sont calculés ainsi :

- Q = Embedding × Wq
- K = Embedding × Wk
- V = Embedding × Wv

Étape 2 - Calcul des scores d'attention

Pour chaque token, on calcule la similarité entre sa Query et les Keys de tous les mots de la séquence en faisant un produit scalaire.

Ce score est ensuite divisé par la racine carré de la taille d'un vecteur (dans "All You Need Is Attention" la valeur choisie est 8 car les vecteurs possèdent 64 dimensions). Cette étape permet d'éviter que les valeurs deviennent trop grandes ce qui stabilise l'entraînement du modèle.

Puis, on applique une fonction Softmax à ces scores pour les normaliser : cela transforme les scores en valeurs positives comprises entre 0 et 1, dont la somme est égale à 1.

Ces poids d'attention indiquent dans quelle mesure chaque mot influence le mot en cours de traitement. Un mot est souvent le plus influencé par lui-même, mais d'autres mots pertinents peuvent aussi avoir un poids élevé.

Une fois que les poids d'attention ont été calculés, chaque Value est multipliée par le score d'attention correspondant (le poids obtenu après Softmax). Cela permet au modèle de concentrer l'attention sur les valeurs importantes et de réduire l'impact des tokens moins pertinents.

Après cette multiplication, on fait la somme de ces vecteurs pondérés pour obtenir une représentation contextuelle enrichie du token, qui prend en compte l'influence de tous les autres tokens de la séquence.

Ce vecteur est ensuite envoyé aux couches suivantes du modèle.

Feed forward :

Après la couche d'attention, la représentation contextuelle de chaque token passe par une couche Feed-Forward (FFN). Cette couche est un petit réseau de neurones à deux couches linéaires séparées par une fonction non-linéaire. Elle permet au modèle d'apprendre des transformations supplémentaires et complexes sur chaque vecteur.

Add & Norm :

À la sortie de chaque sous-couche principale (comme la Self-Attention ou le Feed Forward), le Transformer applique une opération appelée Add & Norm, qui contient deux étapes :

1. Add (résiduel) : Le vecteur d'entrée d'origine est ajouté à la sortie de la sous-couche. Cela s'appelle une connexion résiduelle. Elle aide à préserver l'information d'origine du token, et facilite le passage du gradient pendant l'entraînement. Grâce à cela, le modèle ne "perd" jamais complètement la signification initiale du token.

2. Norm (normalisation) : Ensuite, le vecteur résultant passe par une normalisation de couche (Layer Normalization). Cette étape ajuste les valeurs pour qu'elles aient une distribution plus stable (moyenne proche de 0 et écart-type proche de 1), ce qui rend l'apprentissage plus stable et rapide. Elle empêche aussi que certaines dimensions ne prennent trop d'importance, et garantit que chaque token soit traité équitablement.

Multi-Head Attention :

Au lieu d'une seule attention, le modèle en calcule plusieurs en parallèle. Chaque "tête" dispose de ses propres matrices de projection pour les queries, keys et values, ce qui lui permet d'apprendre à se concentrer sur différents aspects de la séquence (syntaxe, relations longues, coréférences, etc).
Les résultats de chaque tête sont ensuite concaténés pour former un vecteur global. Ce vecteur est passé à travers une couche linéaire qui projette cette concaténation dans l'espace d'origine (espace vectoriel initial, ici 64) et de fusionner l'information des différentes têtes.

Encodeur / Décodeur Blocs :

L'encodeur transforme la séquence d'entrée en une représentation intermédiaire riche qui capture les relations entre les tokens.

Le bloc encodeur est constitué de plusieurs couches empilées :

- Multi-Head Self-Attention : permet à chaque token de prendre en compte tous les autres pour capturer le contexte global.
- Add & Norm : stabilise et accélère l'entraînement, tout en facilitant le passage du gradient.
- Feed-Forward Layer : applique une transformation non linéaire à chaque token pour enrichir sa représentation.
- Add & Norm

Le Décodeur sert à générer une séquence de sortie, un token à la fois, en s'appuyant sur les tokens déjà générés et sur la représentation produite par l'encodeur si il en a un dans le modèle.

Le bloc Décodeur fonctionne de manière similaire mais utilise :

- Masked Multi-Head Self-Attention : Autorise uniquement l'attention sur les tokens précédents
- Add & Norm
- Cross-Attention (sur les sorties de l'encodeur) : Permet au décodeur de se concentrer sur les informations pertinentes de la séquence d'entrée.
- Add & Norm
- Feed-Forward Layer
- Add & Norm

Génération de la sortie (Pour mon modèle)

La génération de sortie diffère selon les modèles ici je vais seulement parler de celle propre à mon modèle.

Les vecteurs associés à chaque token sont moyennés pour produire un unique vecteur de sortie. Ce vecteur est ensuite transmis à une couche linéaire finale, qui génère un score. Une fonction sigmoïde est alors appliquée pour convertir ce score en une probabilité entre 0 et 1, représentant la probabilité que la formule d'entrée soit satisfiable. Si cette probabilité dépasse un seuil (ici 0.5), le modèle prédit que la formule est SATISFIABLE, sinon INSATISFIABLE.

Comment entraîner un Transformer pour résoudre le SAT

Pour entraîner un modèle Transformer, il faut commencer par définir une architecture adaptée aux besoins. Mon modèle est implémenté en Python à l'aide de la bibliothèque PyTorch. Il se compose d'un bloc d'encodage, avec des vecteurs d'entrée de dimension 64 et utilise 2 têtes d'attention.

L'étape suivante est de générer ou collecter un jeu de données suffisamment volumineux et pertinent, en rapport avec la tâche à accomplir. Dans mon cas, la prédiction d'instances de problèmes SAT. Une fois les données préparées, elles sont utilisées pour entraîner le modèle selon un protocole d'apprentissage. Je détaille ces deux aspects dans les sous-parties suivantes.

Génération de données d'entraînement :

Je génère les données d'entraînement et de test à l'aide d'une fonction aléatoire produisant des formules 3-SAT, via la bibliothèque PySAT. Le nombre de variables est tiré aléatoirement entre 5 et 20, et le nombre de clauses entre 20 et 60.

Pour chaque formule, j'utilise un solveur SAT de PySAT afin de déterminer si elle est satisfiable (SAT) ou non (UNSAT). Je m'assure de générer un nombre équilibré d'instances SAT et UNSAT. Les formules sont enregistrées au format DIMACS dans des fichiers .cnf

Une fois les données générées, j’utilise un script pour détecter les doublons parmi les instances SAT. Chaque formule est transformée en une représentation normalisée, puis son hash est calculé. Si deux formules ont le même hash, elles sont considérées comme des doublons. Dans ce cas, je régénère de nouvelles instances.

J’ai également effectué des tests avec les instances disponibles sur SATLIB, mais cette option s'est révélée peu pertinente car les jeux de données proposés sont trop limités en taille et en diversité pour entraîner un modèle Transformer.

Entrainement du Transformer :

Pour entraîner mon modèle Transformer, j’utilise les données générées précédemment. Elles sont divisées en trois ensembles : un ensemble d'entraînement (80 %), un de validation (10 %) et un de test final (10 %).

L'entraînement se déroule sur un nombre d'époques défini, avec un traitement en mini-lots. À chaque itération, les formules sont converties en séquences de tokens, puis passées au modèle, qui génère une prédiction sous forme de probabilité. Cette sortie est ensuite comparée à l’étiquette réelle de la formule à l’aide d’une fonction de perte.

Les gradients sont rétro propagés, et les poids du modèle sont mis à jour à l’aide d’un l’optimiseur avec un taux d'apprentissage fixe. Après chaque époque, le modèle affiche la perte moyenne sur l’ensemble d’entraînement ainsi que les performances sur l’ensemble de validation. Cette évaluation sur des données que le Transformer n’as pas vues pendant l’entraînement permet de mesurer la capacité de généralisation du modèle.

Le processus continue jusqu’à atteindre le nombre d’époques max, ou jusqu’à ce qu’aucune amélioration ne soit observée sur les performances de validation pendant plusieurs époques consécutives.

À la fin de l'entraînement, on sélectionne et exporte la version du modèle ayant les meilleurs sur les données de validation. On l’évalue ensuite sur l’ensemble des données de test final, afin d’estimer sa performance réelle sur des formules jamais vues. On mesure alors la précision et fiabilité de ses prédictions.

Conclusion et perspective

Conclusion :

Ce stage a permis d'explorer l'application des modèles Transformer à un problème fondamental de l'informatique théorique : le problème SAT. J'ai eu l'opportunité d'étudier et comprendre les modèles Transformers et le problème SAT.

J'ai aussi appris à concevoir et entraîner un modèle Transformer en Python capable de prédire avec succès la satisfiabilité de formules logiques exprimées au format 3-SAT.

Mon modèle atteint un taux de bonnes réponses d'environ 90 % sur l'ensemble de test, ce qui démontre sa capacité à extraire des motifs significatifs à partir des structures logiques des formules.

De plus, le modèle entraîné exportée peut être directement utilisé pour analyser de nouvelles formules SAT au format DIMACS. Un script permet d'automatiser cette prédiction et de générer des heatmaps d'attention interprétables.

Les heatmaps d'attention générée par le transformer nous permettent de visualiser quelles parties de la formule influencent le plus la prédiction du modèle. Cette information peut être exploitée dans une approche hybride, en la combinant avec des solveurs classiques comme DPLL. Par exemple, les branches identifiées comme prometteuses par le modèle peuvent être explorées en priorité par les algorithmes classiques, ce qui ouvre la voie à des solveurs plus efficaces, intelligents et guidés par apprentissage.

Perspective :

Plusieurs pistes d'exploration sont possibles pour continuer ce projet :

Augmentation des données : élargir les données d'entraînement avec des formules plus complexes ou issues de sources réelles, afin de mieux évaluer la robustesse du modèle.

Apprentissage par renforcement : intégrer le Transformer dans un cadre de recherche arborescente où il guide dynamiquement les décisions de branchement. Le modèle pourrait être entraîné par renforcement pour optimiser la vitesse ou la profondeur de résolution.

Interprétabilité avancée : pousser l'analyse des attention maps pour comprendre quelles structures logiques ou patterns déclenchent certaines décisions. Cela peut aussi permettre d'améliorer la confiance dans les prédictions du modèle, en expliquant son raisonnement.